

Caracal-1 · plano

Modelo especialista em ciberseguranca de 3 bilhoes de parametros. 5 fundadores. Pool aberto. Começar simples (v0), evoluir em fases. Atualizado 23 jun 2026.

INDICE

Repo · [iterate-labs-ai/caracal-1](#)

Repo · [iterate-labs-ai/caracal-cyber](#) (sandbox + eval)

Issues T1-T10 abertas

1. Visao do projeto
2. Arquitetura do modelo
3. Como o sistema funciona
4. Roteiro v0 → v1 → v2 → v3
5. v0 MVP · o que ja vamos fazer
6. Datasets v0
7. Stack tecnico
8. Tarefas v0 (10)
9. Contas (5 Kaggle + 5 Colab)
10. Sessoes de treino
11. Setup passo a passo (cada fundador)
12. Como vamos trabalhar juntos
13. Como rodar o treino na sua sessao
14. Apos v0 funcionar
15. Glossario

1. Visao do projeto

Caracal-1 é um modelo de linguagem especialista em cibersegurança. Recebe código em C/C++/Rust/Python e gera prova de conceito (PoC) que dispara vulnerabilidade. Também gera o patch que corrige. Base é Qwen2.5-Coder-3B-Instruct (Apache 2.0).

O modelo final tem 5 módulos LoRA especialistas sobre um Caracal Base compartilhado. Cada módulo faz uma parte: scout codebase, gerar hipóteses, escrever PoC, validar, patchar. Conversam entre si via protocolo estruturado.

Diferencial vs frontier (Mythos, GPT-5): pequeno (3B), barato por zero-day, open weight. Roda em Kaggle T4 grátis. Cabe no celular eventualmente.

Mercado: AI cibersegurança Brasil cresce 23% ao ano, fastest LATAM. Anthropic lançou Mythos em mar 2026 mas frontier-only. Gap claro pra small specialist open weight.

2. Arquitetura do modelo

2.1 Caracal Base 3B (fundação)

Transformer decoder-only de 3.09 bilhões de parâmetros. 36 camadas. Atenção agrupada com 16 cabeças de consulta para 2 cabeças de chave-valor. Ativação SwiGLU. RMSNorm. RoPE base 1.000.000. Contexto 32K tokens.



Figura 1. Pipeline de inferência. Texto entra como tokens, passa pelos 36 blocos de decoder, sai como distribuição de probabilidade.

2.2 5 módulos LoRA especialistas (em cima do Base)

Cada módulo é LoRA $r=32$ sobre Caracal Base. Apenas 130MB cada. Caracal Base fica congelado, só os adaptadores treinam.

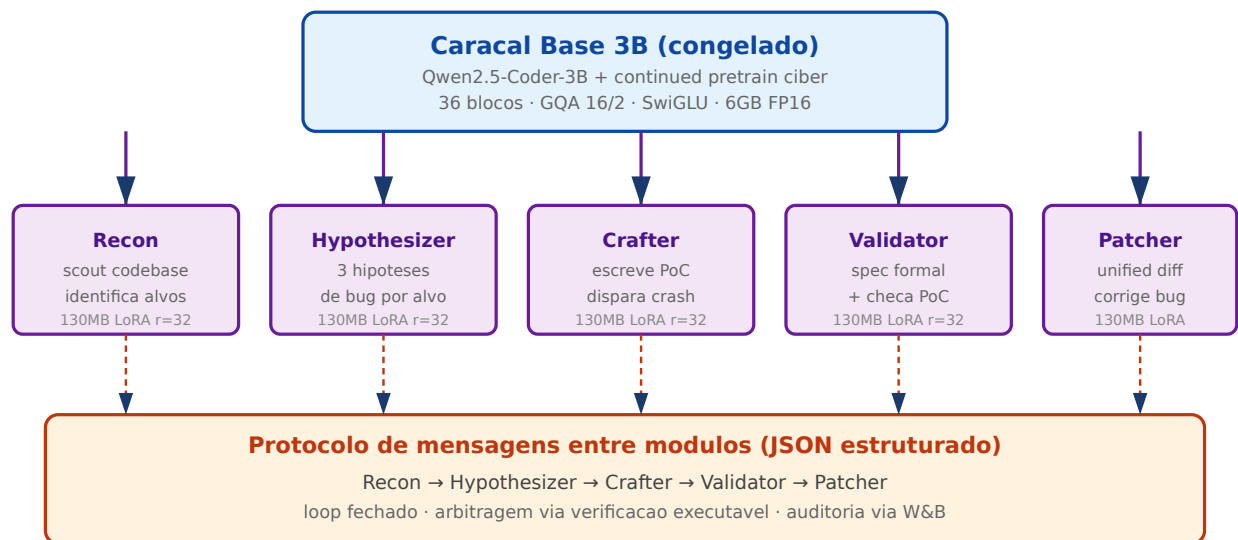


Figura 2. Caracal Base + 5 LoRAs especialistas. Backbone congelado, adapters trocam em microsegundos por slot.

2.3 Total parametros

COMPONENTE	PARAMETROS	DISCO
Caracal Base 3B (heranca Qwen)	3.09 bilhoes	6 GB FP16
5 LoRA adapters (r=32 alpha=64)	5 × 33 milhoes = 165 milhoes	5 × 130MB = 650 MB
Total	~3.26 bilhoes	~6.6 GB

3. Como o sistema funciona (loop completo)

Dado um codebase (ex: libtiff 50K linhas), Caracal-1 produz pares (PoC, patch). Loop completo:

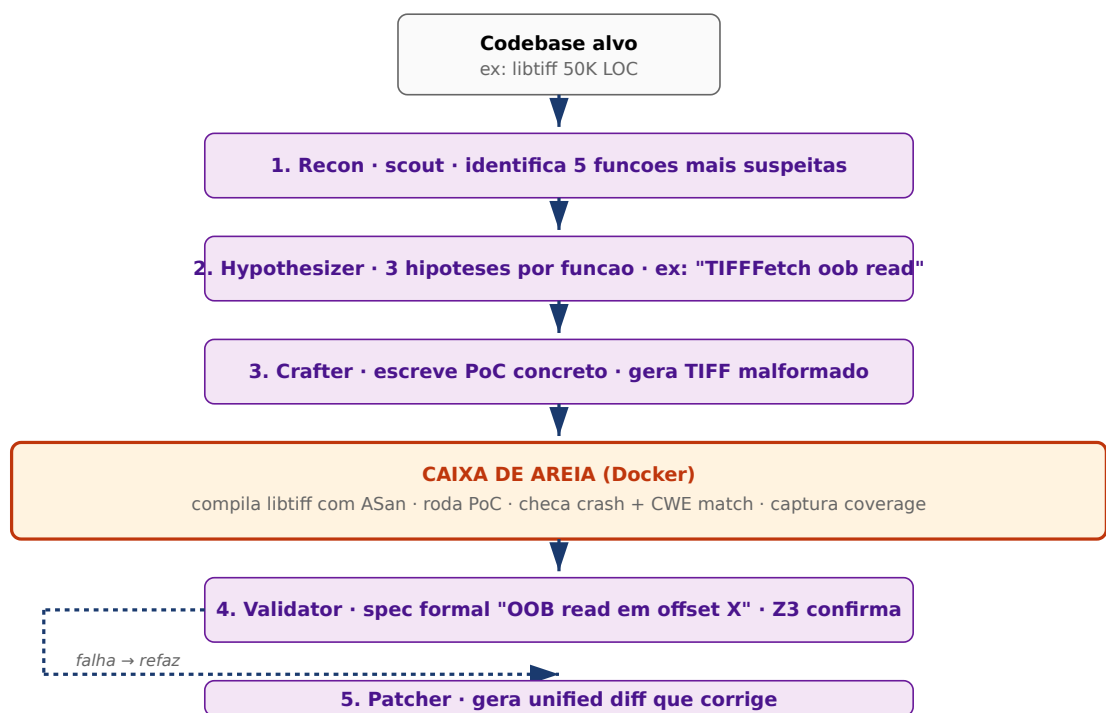


Figura 3. Loop end-to-end. Codebase entra, par (PoC, patch) sai. Caixa de areia verifica execucao real.

4. Roteiro v0 → v1 → v2 → v3

Plano em 4 fases. Cada uma adiciona complexidade. NAO tudo de uma vez.

VERSAO	O QUE TEM	O QUE NAO TEM AINDA	QUANDO	CUSTO
v0 Caracal Base	Qwen Coder 3B + 3 datasets HF · LoRA SFT continued pretrain · roda inferencia simples	5 modulos, sandbox, RL, archive, verificacao	2 semanas (Sprint 0)	\$0
v1 Multi-modulo	5 LoRA modulos (Recon, Hypothesizer, Crafter, Validator, Patcher) · protocolo entre eles	Sandbox de execucao, RL com recompensa verificavel	+2 semanas (Sprint 1)	\$0

v2 Sandbox + RL	Caixa de areia Docker (Modal CPU) · recompensa verificavel · GRPO/DAPO · Darwin Godel archive	Recursive harness, embodied gdb, SMT Z3, SAE	+3 semanas (Sprint 2-3)	~\$300/mes Modal
v3 Recursive + ship	6 loops self-improving · zero-day hunt em 30 OSS held-out · paper arXiv · Caracal-XS edge	—	+1 semana (Sprint 4)	~\$30k total v1 ship

Estamos em v0. Foco: provar que o pipeline de treino funciona. Modelo carrega, treina, salva, roda inferencia. So depois adiciona modulos, sandbox, RL.

5. v0 MVP · o que ja vamos fazer

Treinar Caracal Base 3B v0 em 2 semanas com recursos gratis. Provar pipeline. Avaliar.

O QUE	DETALHE
Base	Qwen2.5-Coder-3B-Instruct (Apache 2.0)
Treino	Continued pretrain SFT com LoRA r=32 alpha=64
Framework	Unsloth + TRL SFTTrainer
Datasets	3 publicos HuggingFace · ~500M tokens (PrimeVul + BigVul + DiverseVul)
Passos	~20.000 passos
Tempo	~25h em Kaggle T4 · 3 sessoes de ~9h
Output	<code>iterate-labs/caracal-base-3b-v0</code> no HF Hub
Avaliacao	50 vulns CyberGym subset · comparar com Qwen base zero-shot
Custo	\$0 (Kaggle + HF + W&B tudo gratis)

6. Datasets v0

3 datasets ja no HuggingFace. Carrega via `load_dataset()` . Zero scraping.

DATASET	TAMANHO	HF	CONTEUDO
PrimeVul	~150M tokens	<code>secmlr/PrimeVul</code>	~7K funcoes C/C++ vulneraveis curadas
BigVul	~200M tokens	<code>bstee615/bigvul</code>	~11K CVEs de 348 projetos open source
DiverseVul	~150M tokens	<code>bstee615/diversevul</code>	~349K funcoes C/C++ vuln + non-vuln

Total ~500M tokens. Decontamination simples: CVE-ID dedup vs CyberGym 1507 (rapido, ja implementado em `eval/check_decontamination.py`).

7. Stack tecnico

CAMADA	ESCOLHA	POR QUE
Placa de video	Kaggle T4 16GB	Gratis · cabe Qwen 3B sem quantizar
Framework treino	Unsloth + TRL SFTTrainer	2-5x mais rapido que TRL puro · 80% menos memoria · notebooks Kaggle oficiais
Adapter	LoRA r=32 alpha=64	~33M parametros treinaveis (1.1% do total) · 130MB por adapter
Precisao	BF16 base + FP16 adapter	Estabilidade · cabe T4
Storage pesos	HuggingFace Hub	Gratis ate 100GB · revisions imutaveis tipo git commits

Tracking	Weights and Biases gratis academic	Loss curve preserve entre sessoes (resume='allow')
Avaliacao	CyberGym subset 50 vulns + HumanEval+ regression	Numero pra comparar versoes
Inferencia futura	vLLM PagedAttention	Padrao industria · suporta LoRA hot-swap

O que NAO usar agora (volta nas fases v1+)

RECURSO	CUSTO EVITADO V0	VOLTA EM
Modal CPU sandbox (Docker hardened)	\$300/mes	v2 quando precisar rodar PoC
Colab Pro+ subscriptions	\$50/mes × N	Avaliar so se Kaggle T4 demorar demais
Wasabi storage S3	\$50/mes	So se passar 100GB no HF
NVD + Project Zero scraping	2 semanas dev	v1 se precisar mais tokens
5 modulos LoRA especialistas	complexidade	v1 apos Base funcionar
verl + DAPO GRPO	setup pesado	v2 quando RL com recompensa verificavel
Darwin Godel archive + MAP-Elites	dev pesado	v2/v3
Embodied gdb live (Recon)	2 semanas dev	v3
SMT Z3 bounded (Validator)	dev pesado	v3
Recursive harness 6 loops	meses	v3

8. Tarefas v0 (10 caminho critico)

T1 5 contas Kaggle phone-verified + GPU T4 ativada

CRITICA

Cada fundador 1 conta. Phone verify obrigatorio pra placa de video. Ativar T4 em /settings.

T2 Contas HuggingFace + W&B + Kaggle Secrets configurados

CRITICA

HF e W&B contas criadas, convites aceitos na org iterate-labs. HF_TOKEN (write) + WANDB_API_KEY em Kaggle Secrets de cada conta.

T3 Inspeccionar os 3 datasets HF

MUST

Rodar `load_dataset()` em PrimeVul, BigVul, DiverseVul. Documentar schema (qual campo de texto, tamanho real, idioma). 2h dev. Result em `data/dataset_inspection.md`.

T4 Smoke test do continued_pretrain.py (100 passos)

MUST

Rodar `train/continued_pretrain.py` com 100 passos em Kaggle T4. Confirma que treino comeca, loss desce, push HF Hub funciona. ~30min.

T5 Decontamination CVE-ID dedup vs CyberGym

MUST

Rodar `eval/check_decontamination.py` contra os 3 datasets. Sair com lista de CVE IDs a remover. Salvar em `data/decontamination/cve_blocklist.json`.

T6 Schedule 3 sessoes em SCHEDULE.md

MUST

Ja documentado. Confirmar quem pega sessao 1, 2, 3. Default: Pedro / Arthur / Vitor.

T7 Sessao 1 · primeiros 7000 passos

CRITICA

Pedro · seg 24 jun 14h-23h BRT. Comeca do zero. Push revision step-7000.

T8 Sessao 2 · continua 7000 → 14000

MUST

Arthur · ter 25 jun 09h-18h. Resume from step-7000. Push step-14000.

T9 Sessao 3 · finaliza · publish caracal-base-3b-v0

MUST

Vitor · ter 25 jun 19h-04h. Resume from step-14000. Push final `iterate-labs/caracal-base-3b-v0`.

T10 Avaliacao basica · 50 vulns CyberGym

MUST

Kevin ou Alexandre. Comparar Caracal Base 3B v0 vs Qwen Coder 3B zero-shot em 50 vulns. Documentar em `infra/baselines/v0_eval.md`. Decisao: continuar ou repensar.

9. Contas (5 Kaggle + 5 Colab + extras Pedro)

CONTA	FUNDADOR	FUNCAO
Kaggle 1	Pedro (PAMF2)	sessao 1 (segunda)
Kaggle 2	Arthur (arturpn1)	sessao 2 (terca manha)
Kaggle 3	Vitor (VitorScrt)	sessao 3 (terca noite)
Kaggle 4	Kevin (dev-knz)	avaliacao + backup
Kaggle 5	Alexandre (aletlucas)	avaliacao + backup
Colab 1	Pedro	experimentos paralelo
Colab 2	Arthur	experimentos paralelo
Colab 3	Vitor	experimentos paralelo
Colab 4	Kevin	experimentos paralelo
Colab 5	Alexandre	experimentos paralelo
Colab extras × 4	Pedro	overflow capacity

CAPACIDADE	TOTAL
Kaggle T4 garantido	5 contas × 20h/sem = 100h/sem
Colab T4 esporadico	5 + 4 = 9 contas × ~3h/dia = ~189h/sem (preempted comum)
Total disponivel	~290h/sem T4
Necessario v0	~25h T4 total

Sobra muita capacidade. 25h treino em 290h disponiveis = 8.6% da nossa capacidade. Resto pra experimentos, avaliacao, smoke tests, backup.

10. Sessoes de treino · 5 sessoes de 5h (uma por fundador)

Cada fundador faz 1 sessao de 5h. 25h treino total. Pesos salvos como Kaggle Datasets publicos (sem HF, sem W&B, sem Secrets).

SESSAO	FUNDADOR	QUANDO BRT	STEPS	RESUME KAGGLE DATASET	OUTPUT KAGGLE DATASET
1	Pedro (PAMF2)	seg 24 jun 14h-19h	0 → 4000	(do zero)	pamf2/caracal-base-step4000
2	Arthur (arturpn1)	seg 24 jun 20h-01h	4000 → 8000	pamf2/caracal-base-step4000	arturpn1/caracal-base-step8000
3	Vitor (VitorScrt)	ter 25 jun 09h-14h	8000 → 12000	arturpn1/caracal-base-step8000	vitors crt/caracal-base-step12000
4	Kevin (dev-knz)	ter 25 jun 15h-20h	12000 → 16000	vitors crt/caracal-base-step12000	dev-knz/caracal-base-step16000

5	Alexandre (aletlucas)	ter 25 jun 21h- 02h	16000 → 20000	dev-knz/caracal- base-step16000	aletlucas/caracal- base-3b-v0 (FINAL)
---	--------------------------	---------------------------	---------------------	------------------------------------	---

5 sessoes × 5h = 25h treino. Cada um pega 5h da quota 20h/sem (sobra muita). T10 avaliacao apos sessao 5 · qualquer fundador disponivel pega.

Storage: Kaggle Datasets publicos. Sem auth complicada. Cada fundador cria seu dataset, proximo baixa via Kaggle CLI. Sem HuggingFace, sem Weights and Biases, sem Kaggle Secrets.

11. Setup passo a passo (cada fundador)

Cada fundador faz isso uma vez, demora ~30 minutos no total. Faz logo pra estar pronto segunda 24 jun.

Passo 1 · GitHub (~3 min)

1. Aceitar convite em github.com/orgs/iterate-labs-ai/invitations
2. Se nao chegou convite, confirmar com Pedro qual handle GitHub voce usa
3. Status checa em github.com/orgs/iterate-labs-ai/people
4. Instalar GitHub CLI `gh` se nao tiver:

```
# Ubuntu/Debian
sudo apt install gh

# Mac
brew install gh

# Windows (PowerShell)
winget install --id GitHub.cli

# Login
gh auth login
# escolher: GitHub.com → HTTPS → Login with web browser
```

Passo 2 · Conta Kaggle phone-verified + GPU T4 (~10 min)

1. Criar conta em kaggle.com/account/login ou logar com Google
2. Phone verify obrigatorio (necessario pra ter GPU):
 - kaggle.com/settings · Phone verification
 - Pode demorar ~5 min pro SMS chegar
3. Ativar GPU T4 em kaggle.com/settings · aceitar terms de uso

Passo 3 · Kaggle CLI local (~2 min)

Pra rodar treino do PC e baixar checkpoints entre sessoes:

```
pip install kaggle
```

Baixar `kaggle.json` em kaggle.com/settings · botao "Create New API Token":

```
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json

# Testar
kaggle datasets list
```

So isso. SEM HuggingFace, SEM Weights and Biases, SEM Secrets. Pesos vivem como Kaggle Datasets publicos.

Passo 4 · Clone repo localmente (opcional)

Quem vai mexer em codigo (alterar scripts, fazer PR), clonar local:

```
# Clone branch dev (default)
git clone https://github.com/iterate-labs-ai/caracal-1
cd caracal-1

# Confirmar esta em dev
git branch
# * dev

# Instalar deps localmente (so se for mexer em codigo)
python -m venv .venv
source .venv/bin/activate # Linux/Mac
# .\.venv\Scripts\activate # Windows
pip install -r requirements.txt
```

Checklist final

ITEM	OK
Aceitei convite GitHub iterate-labs-ai	☐
Conta Kaggle phone-verified	☐
GPU T4 ativada em Kaggle settings	☐
Kaggle CLI instalado + <code>kaggle.json</code> em <code>~/.kaggle/</code>	☐
Clonei repo caracal-1 branch dev	☐

Troubleshooting

PROBLEMA	SOLUCAO
Phone verify Kaggle nao chega	Esperar 10 min · tentar outro numero · usar Google login
Kaggle nao da GPU T4 disponivel	Esperar algumas horas · pode ser quota momentanea
<code>kaggle datasets list</code> da erro auth	Conferir <code>kaggle.json</code> em <code>~/.kaggle/</code> com permissao 600

12. Como vamos trabalhar juntos

Pool aberto. Sem dono. 5 fundadores, mesmas permissoes. Caminho real do trabalho diario:

12.1 Branches no GitHub

BRANCH	PRA QUE	QUEM MODIFICA
<code>main</code>	Releases estaveis (futuro)	So PR de dev quando v0 ship
<code>dev</code>	Default · onde tudo mergeia	Todos via PR

book-slot-N	PR pequena pra bookar slot Kaggle no SCHEDULE	Quem ta bookando
tN-curta-descricao	Feature branch pra trabalhar em tarefa T1, T2, etc	Quem claim a issue

12.2 Workflow diario

Manha

1. Abrir [issues](#)
2. Procurar issue com label `sprint-0` + sem assignee · prioridade `CRITICA` antes de `MUST`
3. Comentar `claim` na issue · GitHub auto-assigna voce
4. `git checkout -b tN-curta-descricao dev`
5. Trabalhar

Durante o trabalho

- Commits atomicos com mensagens curtas: `git commit -m "add X"`, `"fix Y"`
- Push regular: `git push -u origin tN-curta-descricao`
- Pergunta no grupo quando travar

Quando terminar

1. `gh pr create --base dev --title "T3 inspect datasets" --body "Closes #N"`
2. Pedir review de outro fundador no grupo (Discord/WhatsApp)
3. Apos approve, merge via squash
4. Comentar `done` na issue, fechar
5. Voltar pra manha e pegar proxima

12.3 Bookar slot Kaggle (passo concreto)

```
git checkout -b book-slot-1 dev
# editar SCHEDULE.md
# trocar "pending" por "in-progress · PAMF2 · 2026-06-24 14:00"
git add SCHEDULE.md
git commit -m "book slot 1 Pedro segunda 14h"
git push -u origin book-slot-1
gh pr create --base dev --title "book slot 1" --body ""
```

```
# self-merge se nao conflito
gh pr merge --squash
```

12.4 Quem faz o que (Sprint 0)

Pool aberto mas pra MVP funcionar em 2 semanas, dividir assim (cada um pode pegar mais conforme termina). Todas as 10 issues ja foram criadas no repo · [clica pra abrir](#).

FUNDADOR	TAREFAS INDICATIVAS (CLIQUE)
Pedro (PAMF2)	T1 , T2 , T6 , T7 sessao 1
Arthur (arturpn1)	T1 , T2 , T3 , T8 sessao 2
Vitor (VitorScrt)	T1 , T2 , T4 , T9 sessao 3
Kevin (dev-knz)	T1 , T2 , T5 , T10 avaliacao
Alexandre (aletlucas)	T1 , T2 , T3 , backup sessoes

12.5 Branches pre-criadas

Pra cada tarefa Sprint 0 ja tem branch pronta. Quem claim a issue tambem ja tem branch dele:

BRANCH	ISSUE	PRA QUE
t1-work	#18 T1	5 contas Kaggle
t2-work	#19 T2	HF + W&B + Secrets
t3-work	#20 T3	Inspecionar datasets
t4-work	#26 T4	Smoke test 100 passos
t5-work	#21 T5	Decontamination CVE-ID
t6-work	#22 T6	SCHEDULE.md
t7-work	#23 T7	Sessao 1 treino
t8-work	#24 T8	Sessao 2 treino

t9-work

#25 T9

Sessao 3 treino

t10-work

#17 T10

Avaliacao final

```
# Pegar branch da tarefa que voce vai fazer
git fetch origin
git checkout t3-work # exemplo: vou fazer T3
# trabalhar
git add data/dataset_inspection.md
git commit -m "add dataset inspection"
git push
gh pr create --base dev --title "T3 inspect datasets" --body "Closes #20"
```

Quem termina primeiro pega tarefa nova do pool. Backup pega sessao se quem ia rodar nao conseguir.

12.5 Sincronizacao do time

QUANDO	COMO	O QUE
Diario async	Grupo Discord/WhatsApp	"to em T3" · "T3 done, claim T4" · "sessao 1 step-2000 ok, loss 1.8"
Domingo 23 jun 20h BRT	Call kickoff Meet/Zoom 30min	Confirmar todos setados · review SCHEDULE · qual duvida no codigo
Quarta 26 jun 10h BRT	Call sync 20min	Caracal Base 3B v0 publicado? Avaliacao? Decide v1.
Antes de promover ckpt	2 fundadores reviewam	Suite eval + numeros + push HF Hub final

12.6 Como falar com o time

SITUACAO	ONDE
Codigo nao roda	Grupo do time + screenshot do erro
Sessao Kaggle preempted	Grupo do time + ultimo step que salvou
Quero pegar tarefa que ja tem assignee	Pergunta no grupo se pessoa ja terminou

Achei bug critico

Issue urgente label `blocker` + sinaliza grupo

Sandbox attack test falhou (futuro)

PARA TUDO. 2 reviewers + ADR. Discord blocker.

Modal cobrou caro (futuro v2)

Grupo · pausa sandbox · revisao orcamento

12.7 Documentos importantes do repo

ARQUIVO	O QUE TEM
<code>README.md</code>	Visao geral + status atual Sprint 0
<code>SETUP.md</code>	Setup passo a passo (espelha secao 11 deste plano)
<code>HOWTO.md</code>	Workflow diario detalhado
<code>SCHEDULE.md</code>	Slot booking de placa de video
<code>CONTRIBUTING.md</code>	PR style + KERNEL-D rules (futuro)
<code>docs/architecture.md</code>	Arquitetura modelo Caracal
<code>docs/glossary.md</code>	Termos por extenso
<code>docs/plan/iterate_labs.html</code>	Este documento (versao no repo)
<code>train/notebooks/kaggle_continued_pretrain.ipynb</code>	Notebook plug-and-play pra treino
<code>train/continued_pretrain.py</code>	Script Python real do treino
<code>eval/check_decontamination.py</code>	Pipeline 3 camadas de decontamination

13. Como rodar o treino na sua sessao

12.1 Abrir Kaggle, novo notebook

Importar `train/notebooks/kaggle_continued_pretrain.ipynb` do repo OU colar direto:

```
!git clone https://github.com/iterate-labs-ai/caracal-1.git
%cd caracal-1
!git checkout dev
!pip install --quiet 'unsloth[kaggle-new] @ git+https://github.com/unslothai/unsloth.git'
!pip install --quiet trl peft datasets huggingface_hub wandb pyyaml

from kaggle_secrets import UserSecretsClient
import os
secrets = UserSecretsClient()
os.environ['HF_TOKEN'] = secrets.get_secret('HF_TOKEN')
os.environ['WANDB_API_KEY'] = secrets.get_secret('WANDB_API_KEY')
!huggingface-cli login --token $HF_TOKEN --add-to-git-credential
!wandb login $WANDB_API_KEY

# EDITAR PRA SUA SESSAO:
SESSION = 1
RESUME = None # None se sessao 1; senao 'step-7000'
OUTPUT = 'step-7000'
STEPS = 7000

import subprocess, sys
resume_arg = ['--resume-from', f'huggingface://iterate-labs/caracal-base-3b-v0@{RESUME}'] if RESUME else []
subprocess.run([sys.executable, 'train/continued_pretrain.py',
                '--config', 'train/configs/caracal_base_3b.yaml',
                *resume_arg,
                '--steps-to-run', str(STEPS),
                '--output', './ckpt-out',
                '--hf-revision-out', OUTPUT], check=True)
```

12.2 Apos sessao terminar

1. Verificar revision em `huggingface.co/iterate-labs/caracal-base-3b-v0`
2. PR pequena editando SCHEDULE.md: trocar `pending` por `done · step-XXXX`
3. Sinalizar grupo do time
4. Proximo do relay configura RESUME = sua revision e roda

13. Apos v0 funcionar

Reuniao do time. Decisao baseada em avaliacao concreta.

RESULTADO V0

DECISAO

Score sobe vs Qwen base (mesmo 2-5 pontos)	Continuar v1 · adicionar 5 modulos LoRA + protocolo entre eles
Score nao move	Repensar: talvez precisa mais dados, mais epocas, diferente learning rate
Score cai	Debug: contaminacao mascarando? Hyperparam errado? Decontamination falhou?
Treino quebra	Bug script · debug + retry

So apos v0 funcionar a gente considera:

- **v1** 5 modulos LoRA especialistas (Recon, Hypothesizer, Crafter, Validator, Patcher) + protocolo de mensagens entre eles
- **v2** Caixa de areia Docker em Modal CPU + recompensa verificavel + GRPO/DAPO RL + Darwin Godel archive
- **v3** Recursive harness 6 loops + embodied gdb + SMT Z3 + zero-day hunt em 30 OSS held-out + paper arXiv

14. Glossario

TERMO	SIGNIFICADO
Transformer decoder-only	Arquitetura padrao de LLM moderno. So decoder (sem encoder). Familia Llama/Qwen/DeepSeek.
GQA (Grouped Query Attention)	Atencao agrupada. N cabecas de consulta para M cabecas de chave-valor ($M < N$). Reduz memoria KV cache.
SwiGLU	Ativacao do MLP em modelos modernos (Llama 3, DeepSeek, Qwen). Combina linear com sigmoide.
RMSNorm	Normalizacao baseada na raiz da media dos quadrados. Mais barato que LayerNorm.
RoPE	Rotary Position Embedding. Embedding posicional baseado em rotacao.

LoRA r=32	Low-rank adapter posto 32. Treina ~1% dos parametros. Salva 130MB em vez de 6GB.
SFT (Supervised Fine-Tuning)	Treino com pares (input, output esperado).
Continued pretrain	Continua o treino com corpus novo antes do SFT. Injeta capability nova mantendo base.
GRPO / DAPO	Algoritmos de aprendizado por reforco do DeepSeek-R1 e variantes 2026.
HF Hub	HuggingFace Hub. Plataforma pra publicar modelos. Revisions imutaveis.
Unsloth	Framework treino otimizado. 2-5x mais rapido + 80% menos memoria que TRL puro.
vLLM	Motor de inferencia alta vazao com PagedAttention.
PoC (Proof of Concept)	Codigo curto que reproduz uma vulnerabilidade especifica.
CVE	Common Vulnerabilities and Exposures. Catalogo publico (CVE-2024-12345).
CWE	Common Weakness Enumeration. Classificacao de tipos de bug (CWE-119 buffer overflow).
ASan / UBSan / MSan	Sanitizers do compilador. Detectam acesso fora de limite, undefined behavior, memoria nao init.
OSS-Fuzz	Servico Google fuzzing continuo em projetos open-source criticos.
CyberGym	Benchmark Berkeley ICLR 2026. 1507 vulnerabilidades reais de 188 projetos.
Darwin Godel Machine	Arquivo evolutivo de checkpoints. Sakana AI 2025.
MAP-Elites	Quality-diversity. Guarda melhor por combinacao de comportamentos.

SMT Z3

Satisfiability Modulo Theories. Solver de logica primeira ordem da Microsoft.

SAE (Sparse
Autoencoder)

Decompoe ativacoes em features interpretaveis. Anthropic publicou pra Claude.

Repo: github.com/iterate-labs-ai/caracal-1 · branch `dev`

Plano em fases. Comeca v0 (gratis, 2 semanas). Apos funcionar, decide v1, v2, v3.